EDGE COMPUTING BASED PREDICTIVE MAINTENANCE MIDDLEWARE FOR INDUSTRIAL IOT

BLESSING NGONIDZASHE MUSUNGATE [1]
[1] YAŞAR UNIVERSITY, Department of Computer Engineering,
bnmusungate@gmail.com
+90 538 610 6871
ORCID ID: 0009-0003-3738-5901

AHMET TUNCAY ERCAN [2]
[2] Assoc.Prof. YAŞAR UNIVERSITY, Department of Management Information Systems,
tuncay.ercan@yasar.edu.tr
+90 232-570 8241
ORCID ID: 0000-0003-0014-5106

Abstract

Edge Computing and predictive maintenance have become important terms in the research area of the Industrial Internet of Things. Edge computing is an evolution of cloud computing that brings the computing and storage resources of a cloud closer to the devices thus reducing problems of latency and high data influx. Predictive maintenance (PdM) is a type of maintenance that is done just before the equipment stops working with the help of machine learning algorithms. PdM results in less machine downtime and less wastage of device lifetime therefore helping industries to save costs. It follows that a combination of PdM and edge computing is a powerful way in which industries can save money and increase efficiency. In this work we develop an edge computing based predictive maintenance middleware that can be used on an industrial site. The middleware was developed using the Node-RED platform. Node-RED is a platform that allows developers to quickly build internet of things applications using a simple web browser interface. In the platform three different machine learning classification and regression algorithms namely Decision Tree, Random Forest and Support Vector Machines are used. These algorithms predict the quality of a product for the classification problem and predict the remaining useful time of a machine for the regression problem. Our findings show that the Random Forest algorithm has the highest accuracy and fscore for classification and the lowest mean absolute and root mean squared errors for regression. Therefore, Random Forest performs better than Decision Tree and Support Vector Machine algorithms in both classification and regression.

Keywords: Industrial Internet of Things, IIoT, IoT, edge computing, predictive maintenance, machine learning, classification, regression

1. INTRODUCTION

The Internet of Things (IoT) is a system of devices that communicate through the internet without human help. Typically, IoT devices send data to the cloud for storage and processing. IoT has resulted in many devices collecting and sending a lot of data to the cloud. Huge volumes of data have made it more expensive to use cloud services. Furthermore, there are delays (latency) in accessing data from the cloud (end-to-end delay of over 80ms [1]). These challenges can cause serious problems in real-time applications, for example in industry where delays need to be as low as possible. To help solve the problems of high data influx and latency in current IoT, edge computing technology has been developed. This technology extends the cloud computing capabilities (processing and sometimes storage) to the system [2] by bringing the computing resources closer to the devices thus reducing latency and high data influx. In an edge computing architecture, the edge nodes are situated closer to the devices that are generating the data. Edge nodes can be separate devices such as PCs (personal computers) or gateways or, recently, smart PLCs. The actual IoT devices themselves (e.g. sensors, smartphones, PCs) can also act as edge nodes [2]. Edge computing is well suited for applications that have hard real-time constraints i.e., very low latency requirements. Industrial IoT (IIoT) applications are an important example of applications with real-time constraints.

Predictive maintenance (PdM) is a relatively new field of maintenance that aims to conduct maintenance just before the equipment stops working. There are three types of maintenance: reactive, preventive, and predictive maintenance. Reactive maintenance is when maintenance is carried out after a fault occurs in the equipment. However, this approach can result in machine downtimes that can lead to major financial losses in industry. On the other hand, preventive maintenance is when maintenance is carried out long before equipment failure by conducting regular checks. This approach can result in a wastage of equipment lifetime. PdM is different from reactive and preventive maintenance in that it aims to reduce both the downtime and the lifetime wastage of the equipment by predicting the approximate time of machine failure.

There is a good amount of literature about edge computing, artificial intelligence and predictive maintenance. Edge computing can coexist and even cooperate with cloud computing [3]. An approach has been developed whereby training is done on the cloud and trained models are passed on to the edge for use[4]. Another study proposes a two-level edge computing architecture - one layer has cellphone base stations as edge servers and the other layer contains autonomous vehicles as devices with storage and computing resources - for

automated driving services for coordinated content delivery[5]. Machine learning algorithms are used to identify machine stops as well as predicting the product quality in industrial areas including plastic injection molding [6][7]. Calculation of the remaining useful lifetime (RUL) of machines using historical data has become an important research area [8]. Although algorithms like the Random Forest (RF) and Extreme Gradient Boosting (XGBoost) generally perform well in predoctive maintenance they can be imporved by techniques like use Synthetic Minority Over-Sampling Technique (SMOTE) [9]. The Node-RED program has started to see more research interest in its use as a middleware for industrial IoT applications. [10]. A survey that reviewed machine learning (ML) techniques applied in predictive maintenance (PdM) of industrial components between years 2010 and 2020 found out that Support Vector Machine (SVM), Random Forest (RF), and Artificial Neural Networks (ANN) are the most used and most successful ML algorithms in PdM applications[11].

Our work aims to develop a middleware for edge computing in IIoT that can be used on an industrial site for predictive maintenance. The paper is structured as follows: Section 2 describes our middleware solution detailing the steps taken to initialize and execute it. Section 3 discusses our findings from the tests run on the middleware. Section 4 concludes our work and suggests future research directions.

2. EXPERIMENTAL SETUP

2.1. Middleware

2.1.1. Overview

The purpose of this work is to develop a predictive maintenance-based edge computing middleware that can be used on an industrial site. We have chosen plastic injection molding as our industrial case study. Plastic injection molding is a manufacturing process that is widely used in industry because it enables the production of complex plastic objects with high precision and productivity. According to some sources, around 110,000 new injection molding machines are put into operation worldwide every year[12]. Plastic injection molding is a sequential process where the plastic is melted, pressed into the mold, cooled to solidify, and removed from the mold as a three-dimensional shape[8]. Fig 1. shows a pictorial view of a plastic injection molding machine with its important parts labelled [13] and Fig 2. shows its cross section [14].

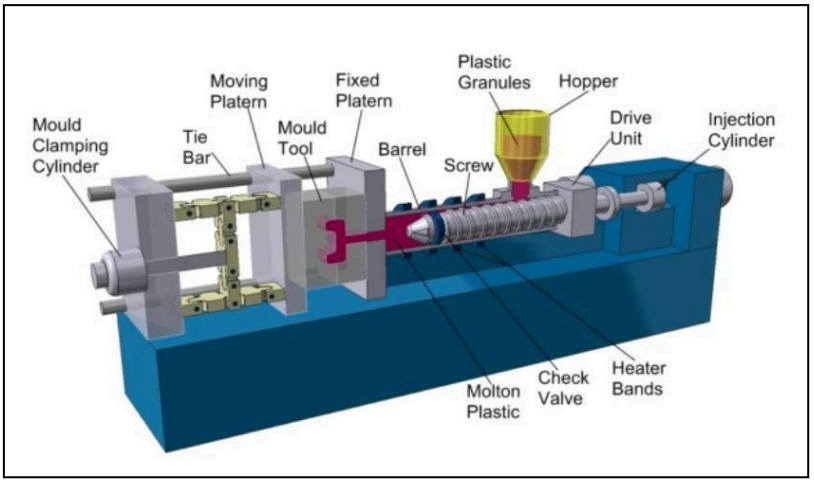


Fig 1. Plastic injection molding machine

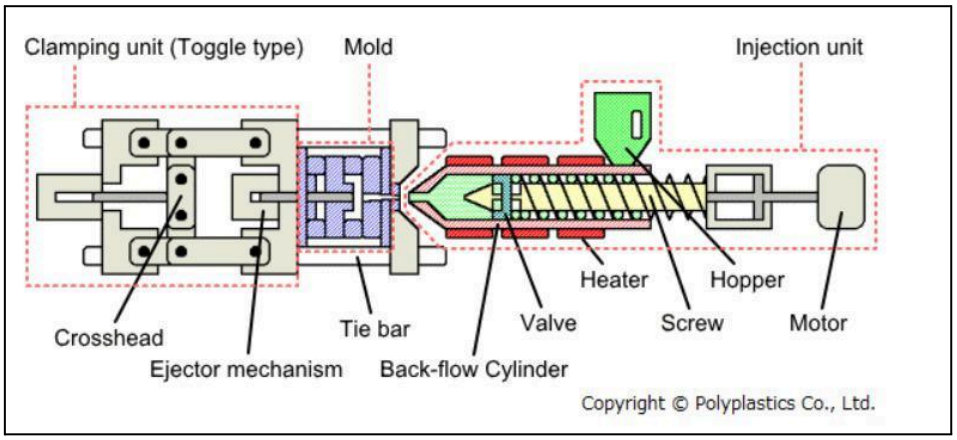


Fig 2. A cross section through a plastic molding machine

In this study incoming data is used to predict quality of the product and then predict when a particular machine could fail based on that data. The data contains process parameters for the plastic injection molding process. These parameters include melt and mold temperatures, cycle time and injection pressure peak value. The data used in this paper was obtained from a research paper [15] that uses actual plastic injection molding data to make machine learning analysis. Fig 3. shows the general flow diagram of the proposed middleware. The sensors generate data and send it to the edge device which processes the data and passes it to the cloud (if necessary) via a webservice. The processor part of the edge device is connected to the database or files to get historical data and applies machine learning algorithms to make decisions.

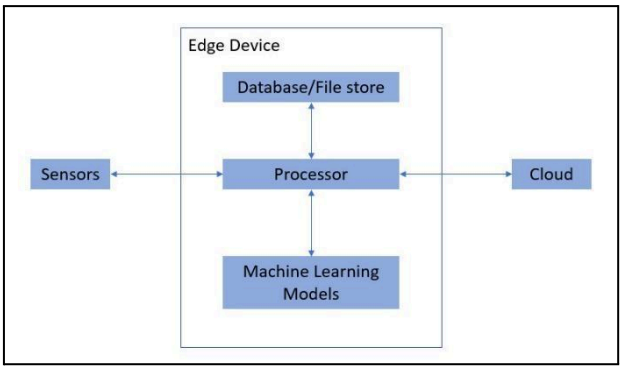


Fig 3. General flow diagram of the proposed middleware

2.1.2. Node-RED Framework

The middleware has been developed using the Node-RED framework. Node-RED is a flow-based programming tool that makes it easy to build event driven applications. The framework uses the Node.js run time environment. It provides a browser-based editor that makes it easy to wire together flows using the wide range of nodes in the palette that can be deployed to its runtime in a single-click [16]. Fig 4. shows the overview of the whole flow of our middleware application in Node-RED displayed in a Chrome browser. The interface can be divided into three sections: left, center and right. The left section of the interface is the command palette from which installed nodes can be dragged and dropped on to the central part. The central part is our work area in Node-RED. This is where the nodes are dragged to and connected. The right side shows information about the configuration and execution of the application. Logs can be shown on this part. Our middleware application is made up of three sections, namely: inputs, edge node and web service. Table 1. describes the function of each section.

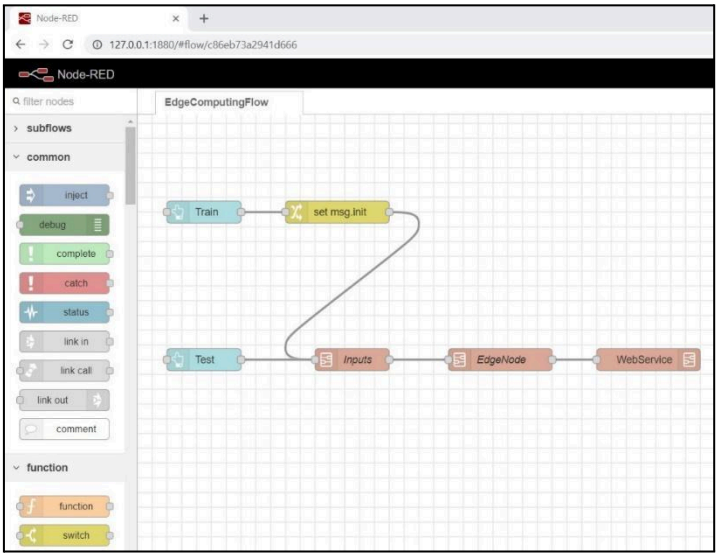


Fig 4. Overview of the Node RED application flow

Table 1. Function of Each Section in the Middleware

Section	Function
Inputs	The inputs section is a part where communication with sensors is done. Communication can be done using extremely lightweight publish/subscribe messaging protocols like the Message Queue for Telemetry Transport (MQTT) protocol . [17].
Edge Node	This is the processing part of the middleware. Also contains machine learning algorithms for data processing.
Web service	Communication with the cloud, if necessary, can be made from this section.

2.2. Methodology

2.2.1 Initialization and Execution

For the middleware to work properly, the edge node needs to be initialized first.

Initialization

During this step the edge node is started and historical data from the database and files is read and processed. Steps taken in the initialization stage include:

- Reading data from the database and files (e.g., rules, historical data)
- Data preprocessing and normalization
- Storing key values temporarily for future reference (e.g., min, max values)
- Initializing and training machine learning models

Execution

This second stage is when the edge node processes incoming data in real-time. When sensor readings are received from the inputs module, these steps are done in the edge node:

- Parsing incoming data to relevant format
- Getting threshold values from temporary storage
- Sending the data to machine learning models to get results
- Predicting results using machine learning algorithms

2.2.2 Datasets

Two datasets are used in this experiment. The first one (Dataset A) contains plastic molding injection products grouped into four different quality classes based on process parameters. This dataset is used for our classification problem. The second dataset (Dataset B) contains remaining useful time (RUL) information for a machine process based on process

parameters for an industrial degradation process. Dataset B is used for our regression problem. It is important to note that to the best of our knowledge we could not find a publicly available plastic injection molding dataset that contains machine remaining useful time as well as timestamps. Therefore, we used Dataset B. 80% of each dataset is used to train the machine learning algorithms and the rest is used for testing.

Classification

Dataset A has 1457 samples and contains actual readings from a plastic injection molding company taken from a plastic molding machine [15]. The dataset classifies products into four different classes based on quality. Quality is the target class label that can take values 1, 2, 3 and 4. Table 5. shows part of Dataset A used in the experiment. The meaning of each possible target class value is explained below:

- 1 - Waste – this product cannot be used because of low quality.
- 2 - Acceptable – this product is of acceptable quality and can be used.
- 3 - Target – this product has the desired quality and is of the standards that the factory wants.
- 4 - Inefficient – this product is of acceptable quality but has used more resources than expected.

The goal of using classification in this experiment was to see how the algorithms can interpret plastic injection molding data by correctly classifying the products. This can be helpful in assisting factory managers to make important decisions, for example about the product quality.

Regression

Dataset B contains 1178 samples of readings that contain a couple of parameters and remaining useful lifetime (RUL) data of a machine for an industrial degradation process [18]. The observed degradation process is the clogging of filters when separating solid particles from gas. RUL is the time before a machine stops working due to a fault. In Dataset B RUL is given in number of machine cycles left before a failure occurs. Table 6. shows part of Dataset B used in the experiment. The goal of using regression in this experiment was to calculate RUL of a machine based on incoming data. RUL is a continuous value and is best handled by a regression approach. RUL is then used to predict machine cycles left. Consequently, this gives an estimated time before a machine can fail.

2.2.3 Machine Learning Algorithms

The edge node in the Node-RED middleware contains a special node called the machine algorithms module. This module makes use of machine learning (ML) algorithms for prediction. Depending on the nature of historical data, available supervised or unsupervised ML algorithms can be used. For supervised algorithms the target class (feature) of the historical data is known and therefore training is straightforward. For unsupervised learning however, the historical data does not have the target attribute and hence the algorithms must find different ways of grouping the data. In this work, we only used supervised learning. Three supervised learning algorithms were used in our experiment, namely: Decision Tree (DT), Random Forest (RF) and Support Vector Machine (SVM). These algorithms were used for both classification and regression. Each algorithm had two separate code bases, one for classification and another for regression. Hence a total of six algorithms were trained and tested. To implement the ML in Node-RED, a collection of libraries for ML in Node.js called ml.js was used [19]. This collection contains DT, RF and SVM as separate modules that can be installed and run independently. The collection also contains special data structures like matrices which are required by DT and RF algorithms. Each algorithm is trained and tested. Fig 5. shows the flow for an RF ML model for both classification and regression algorithms. Fig 6. and Fig 7. show training and testing (execution) for the RF classification algorithm respectively.

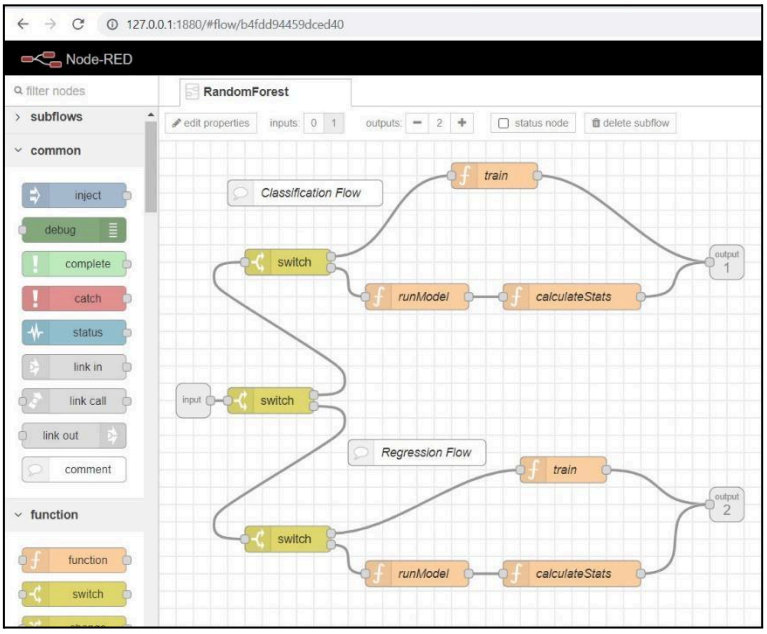


Fig 5. Node RED flow for a Random Forest ML model

```
const { Matrix } = MLMatrix;
const { RandomForestClassifier } = mlRandomForest;

const trainingData = msg.payload;
let featuresDataArray = [];
let targetDataArray = [];

for(let i=0; i<trainingData.length;i++){
    let row = trainingData[i];
    let rowArray = Object.values(row)
    let features = []
    let target;
    for(let j=0; j<rowArray.length;j++){
        let data = rowArray[j]
        if(j==rowArray.length-1)
            target = data
        else features.push(data)
    }
    featuresDataArray.push(features)
    targetDataArray.push(target)
}

const X = new Matrix(featuresDataArray);
const Y = targetDataArray;

const options = {
  seed: 3,
  maxFeatures: 0.8,
  replacement: true,
  nEstimators: 25
};

const classifier = new RandomForestClassifier(options);
classifier.train(X, Y);

flow.set("rfClassificationModel", classifier)
msg.payload ={
    featuresDataArray,
    targetDataArray
}
return msg;
```

Fig 6. Training code for a Random Forest ML algorithm

```

const { Matrix } = MLMatrix;
const testData = msg.payload;
let featuresDataArray = [];
let rowArray = Object.values(testData);
const actual = rowArray.pop()
let features = [];

for(let j=0; j<rowArray.length;j++){
    features.push(rowArray[j])
}

const Xtest = new Matrix([features]);

const classifier = flow.get("rfClassificationModel");
let result = ""
try{
   let resultData = classifier.predict(Xtest);
   result = parseInt(resultData[0])
}catch(e){}

msg.payload = {
    algorithm: "RandomForest",
    result,
    testData,
    actual
}
return msg;

```

Fig 7. Testing code for a Random Forest ML algorithm

2.2.4 Comparison of Algorithms

Algorithms are compared according to their performances when applied to the two datasets. Classification algorithms of DT, RF and SVM are compared in the classification problem using Dataset A. Regression algorithms of DT, RF and SVM are compared in the regression problem using Dataset B.

Classification

The algorithms predict the quality of the product based on incoming data (parameters) using Dataset A. These parameters represent the conditions existing when the product is being made e.g., mold temperature, pressure, cycle length. The predicted quality value is then compared with the actual quality value to determine if an algorithm has made a correct prediction. Accuracy values of the algorithms are compared to determine which algorithm performs better than others. Algorithmic accuracy is calculated using this simple formula in Equation (1)

$$\text{Accuracy} = \frac{Number\ of\ test\ instances\ correctly\ predicted}{Number\ of\ all\ test\ instances\ predicted} \text{ x } 100 \quad (1)$$

Precision, recall and f-scores are calculated for each class for each algorithm. For a class *i*:

- TP_i represents true positives for *i* (when the algorithm correctly predicts class *i*)
- FP_i, represents false positives for *i* (when the algorithm predicts the class is *i* but the actual class is different)
- FN_i represents false negatives for *i* (when the algorithm predicts a class different from *i* but the actual class is *i*)

Precision for class *i* , p_i is calculated as follows:

$$p_i = \frac{TPi}{TPi + FPi} \quad (2)$$

Recall for class *i*, r_i is calculated as follows:

$$r_i = \frac{TPi}{TPi + FNi} \quad (3)$$

F score for class *i*, f_i is calculated from precision and recall as follows:

$$f_i = 2 \text{ x } \frac{pi \; x \; ri}{pi + ri} \quad (4)$$

Regression

The regression algorithms predict the remaining useful lifetime (RUL) based on incoming data (parameters) using Dataset B. The predicted RUL value is then compared with the actual RUL value to determine how close the predicted value is to the actual one. Performance metrics for regression algorithms are different from those of classification algorithms. Here we calculate Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) for each algorithm. Equations (5) and (6) show the calculation of MAE and RSME respectively. N represents the total number of observations; P_i represents the predicted RUL value of observation *i* and A_i represents *i*'s actual RUL value.

$$\text{MAE} = \frac{\sum_{i=1}^{N} |Pi - Ai|}{N} \quad (5)$$

$$\text{RMSE} = \sqrt{\frac{\sum_{i=1}^{N} (Pi - Ai)^2}{N}} \quad (6)$$

3. EXPERIMENTAL FINDINGS

Since we used classification and regression in our work the algorithms have been compared under these two approaches.

3.1 Classification

It took 40.5 seconds to train the three algorithms in Node-RED for the classification problem. Table 2. shows the accuracy results and Table 3. shows the precision, recall and f-scores for the algorithms.

Table 2. Accuracy Results for ML Classificatıon Algorithms

Algorithm	Accuracy (%)
Decision Tree (DT)	92.41
Random Forest (RF)	92.76
Support Vector Machine (SVM)	90.35

Table 3. Precision, Recall and Fscores for ML Classifcation Algorithms

Algorithm	Class	Precision	Recall	F score
Decision Tree (DT)	1	0.937	0.914	0.925
	2	0.910	0.888	0.899
	3	0.898	0.930	0.914
	4	0.946	0.972	0.959
Random Forest (RF)	1	0.973	0.889	0.929
	2	0.892	0.925	0.908
	3	0.898	0.930	0.914
	4	0.946	0.972	0.959
Support Vector Machine (SVM)	1	0.880	0.901	0.890
	2	0.892	0.825	0.857
	3	0.898	0.930	0.914
	4	0.946	0.972	0.959

It can be seen from the results that Random Forest was the best of the three algorithms with an accuracy of 92.76%. Consequently, RF had better f-scores than DT and SVM. DF is the second-best algorithm.

3.2 Regression

The three regression algorithms were trained in a time of 23 seconds. MSE and RSME of the three algorithms are shown in Table 4.

Table 4. MSE and RMSE Results for ML Regression Algorithms

Algorithm	MSE	RMSE
Decision Tree (DT)	4.967	7.787

Random Forest (RF)	3.599	4.598
Support Vector Machine (SVM)	9.263	11.18

Results from the regression problem show that RF performed better than the DT and SVM by having lower MSE and RMSE values. DT was ranked second and SVM had the worst performance.

3.2 Node-RED Dashboard

Node RED has an easy-to-build dashboard layout that enables the user to interact with the program. We used the dashboard to visualize the algorithm performance as well as to execute actions like to train and test the algorithms. Parameters like temperature, pressure and cycle length can also be visualized. Fig 8. shows a screenshot of the dashboard from our program.

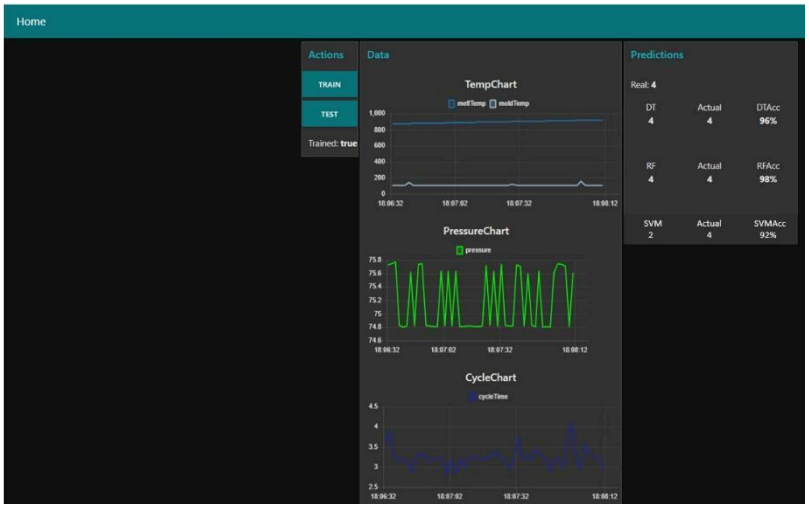


Fig 8. Node RED dashboard view

4. CONLUSION AND FUTURE WORK

In this work an edge computing-based model was developed using the Node RED framework for predictive maintenance in an Industrial Internet of Things (IIoT). Two datasets were used to train and test the machine learning (ML) algorithms namely Decision Tree (DT), Random Forest (RF) and Support Vector Machine (SVM). Dataset A contained data from a real plastic injection machine and was used in the classification problem where the ML algorithms predicted product quality. Dataset B, on the other hand, contained remaining useful lifetime (RUL) data for an industrial degradation process for a machine. This data was used in

the regression problem where the ML algorithms predicted the RUL for the machine based on the incoming data. According to our findings, RF had the best accuracy and f score values for classification. RF also produced the best mean absolute error and root mean squared error values for regression. Hence RF performed better than DT and SVM. DT came in second for both regression and classification. SVM was the worst performer for classification and regression.

As future research directions, we suggest a few ideas. The same experiments in this work can be done with process parameter values in timeseries format as suggested in this article[12]. In addition, more research can be put into analyzing relationships between time and product quality. A comprehensive dataset that contains process parameters, timestamps and maintenance data of the machines can be useful in getting more accurate and realistic results.

Table 5. Part of the Dataset for Quality Prediction

Melt temp (°C)	Mold temp (°C)	Time to fill (s)	Plasticizing time (s)	Cycle time (s)	Closing force (N)	Clamping force peak (N)	Torque peak value (Nm)	Torque mean value (Nm)	Specific back pressure peak value (Bar)	Specific injection pressure peak value (Bar)	Screw position (cm)	Shot volume (cm^3)	Quality
106.141	81.291	6.968	3.19	74.82	910.5	929.9	114.5	105	146.6	917.7	8.85	18.72	2
106.018	80.371	10.764	2.81	75.64	903.8	920.1	117.8	107.8	146.1	926.6	9.02	18.54	3
105.944	81.402	7.228	3.16	74.83	908.6	938.2	120.5	104.9	145.9	917.5	8.85	18.89	1
106.104	81.995	11.128	3.01	75.62	900	917.3	120.5	102.1	145.4	870.6	8.74	18.83	4
147.789	82.099	11.128	2.94	75.65	900.9	915.6	116	104.4	146.4	887.3	8.86	18.71	4
106.273	81.402	6.864	3.19	74.83	886.9	902.7	120.5	106.7	145.8	920.2	8.83	18.73	1

Table 6. Part of the Dataset used for RUL Prediction

Data_No	Differential pressure	Flow_rate	Time	Dust_feed	Bias_type	RUL
20	9.530085	83.26441	68.9	59.11149	4	67.7
4	0	82.32887	1.3	59.11149	2	127.9
19	0	82.78543	21.8	59.11149	1	73
2	271.1121	82.93762	89.7	59.11149	4	40
19	122.1615	82.74316	53.8	177.3131	1	41
1	81.86318	84.98906	100.8	177.3131	3	37.3

REFERENCES

[1] Hassan, N., Yau, K. L. A., & Wu, C. (2019). Edge computing in 5G: A review. IEEE Access, 7, 127276–127289. https://doi.org/10.1109/ACCESS.2019.2938534

[2] Bierzynski, K., Escobar, A., & Eberl, M. (2017). Cloud, fog and edge: Cooperation for the future? *2017 2nd International Conference on Fog and Mobile Edge Computing*, FMEC 2017, 62–67. https://doi.org/10.1109/FMEC.2017.7946409

[3] Cao, K., Liu, Y., Meng, G., & Sun, Q. (2020). *An Overview on Edge Computing Research. 8*. https://doi.org/10.1109/ACCESS.2020.2991734

[4] Calo, S. B., Touna, M., Verma, D. C., & Cullen, A. (2017). *Edge Computing Architecture for applying AI to IoT*. 3012–3016.

[5] Yuan, Q., Zhou, H., Li, J., Liu, Z., Yang, F., & Shen, X. S. (2018). Toward Efficient Content Delivery for Automated Driving Services : An Edge Computing Solution. *IEEE Network*, 32(February), 80–86. https://doi.org/10.1109/MNET.2018.1700105.

[6] Kozjek, D., Kralj, D., Butala, P., Lavrač, N., Stief, P., Dantan, J., Etienne, A., & Siadat, A. (2019). Data mining for fault diagnostics : A case plastic injection molding molding, *Procedia CIRP*, 81, 809–814. https://doi.org/10.1016/j.procir.2019.03.204

[7] Párizs, R.D, Török, D., Ageyeva, T., & Kovács, J.G (2022). Machine Learning in Injection Molding: An Industry 4.0 Method of Quality Prediction. *Sensors 2022*, 22, 2704. https://doi.org/10.3390/s22072704

[8] Aslantas, G., Ozsarac, M., Rumelli, M., & Alaygut, T. (2022). *Prediction of Remaining Useful Life for Plastic Injection Molding Machines Using Artificial Intelligence Methods. 2*(1), 8–15.

[9] Aslantaş G., Alaygut, T., Rumelli, M., Özsaraç M., Bakirli G., Birant, D. (2023). Estimating types of faults on plastic injection molding machines from sensor data for predictive maintenance, *Artificial Intelligence Theory and Applications*, 3(1), 1-11

[10] Stojkic, Z., Bosnjak I., Saravanja L. & Culjak E. (2021). *Predictive Maintenance Supported by IIoT Technologies. January*. https://doi.org/10.2507/32nd.daaam.proceedings.052

[11] Çınar, Z. M., Nuhu, A. A., Zeeshan, Q., & Korhan, O. (2020). *Machine Learning in Predictive Maintenance towards Sustainable Smart Manufacturing in Industry 4 . 0. Ml.*

[12] Bogedale, L., Doerfel, S., & Schrodt, A. (2023). *Online Prediction of Molded Part Quality in the Injection Molding Process Using High-Resolution Time Series*. 1–22.

[13] https://predictabledesigns.com/introduction-to-injection-molding/ (Accessed: 13.06.2023)

[14] https://www.polyplastics.com/en/support/mold/outline/ (Accessed: 13.06.2023)

[15] Polenta, A., Tomassini, S., Sernani, P., Falcionelli, N., Contardo, P., & Dragoni, A. F. (2022). *A Comparison of Machine Learning Techniques for the Quality Classification of Molded Products*.

[16] https://nodered.org/ (Accessed: 25.04.2023)

[17] https://mqtt.org/ (Accessed 27.04.2023)

[18] Hagmeyer, S., Mauthe, F., & Zeiler, P. (2021). *Creation of Publicly Available Data Sets for Prognostics and Diagnostics Addressing Data Scenarios Relevant to Industrial Applications*. 1–20.

[19] https://github.com/mljs (Accessed: 28.04.2023)